EXPERIMENT-11

Aim: Introduction to SQLite database. Set up an SQLite database in your android application. Create a table and perform basic CRUD operations.

Theory:

1. **What is SQLite?**
   SQLite is a lightweight database built into Android.
   - No internet needed
   - Stores data locally (inside the app)
   - Used for apps like contacts, notes, etc.

2. **Create New Android Project**
   1. Open Android Studio
   2. Click New Project
   3. Choose Empty Views Activity
   4. Name: **SQLiteStudentApp**
   5. Language: Java
   6. Click Finish

3. **Create Database Helper Class**
   Right-click → New → Java Class → DatabaseHelper
   Explanation:

- SQLiteOpenHelper → Helps create/manage database
- onCreate() → Runs first time → creates table
- onUpgrade() → Updates database when version changes

4. **Add CRUD Methods**
   Add below methods inside DatabaseHelper class.
   1) **Insert Data (Create)**
      Explanation:
      - ContentValues → stores data in key-value form
      - insert() → inserts row into table
   2) **View Data (Read)**
      Explanation:
      - Cursor → used to read data row by row
      - rawQuery() → executes SQL query
   3) **Update Data**
      This operation modifies the existing data present or stored in tables.
   4) **Delete Data**
      This operation deletes existing record(s) from a table.

5. **Design UI (activity_main.xml)** which consists of code of all widgets present inside app.

6. **MainActivity Code** is responsible for performing all CRUD operations.

Code:

DatabaseHelper.java

activity_main.xml | MainActivity.java | DatabaseHelper.java

```java
package com.example.experiment_11;

import android.content.Context;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

import android.content.ContentValues;
import android.database.Cursor;
no usages
public class DatabaseHelper extends SQLiteOpenHelper {
    1 usage
    public static final String DATABASE_NAME = "Student.db";
    6 usages
    public static final String TABLE_NAME = "student";
    1 usage
    public static final String COL_1 = "ID";
    2 usages
    public static final String COL_2 = "NAME";
    2 usages
    public static final String COL_3 = "MARKS";
    no usages
    public DatabaseHelper(Context context) {
        super(context, DATABASE_NAME,  factory: null,  version: 1);
    }
    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE " + TABLE_NAME +
                   "(ID INTEGER PRIMARY KEY AUTOINCREMENT, NAME TEXT, MARKS INTEGER)");
    }
    10 usages
    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS " + TABLE_NAME);
        onCreate(db);
    }
```

```java
    no usages
    public boolean insertData(String name, String marks) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues contentValues = new ContentValues();
        contentValues.put(COL_2, name);
        contentValues.put(COL_3, marks);
        long result = db.insert(TABLE_NAME, nullColumnHack: null, contentValues);
        return result != -1;
    }
    no usages
    public Cursor getAllData() {
        SQLiteDatabase db = this.getWritableDatabase();
        return db.rawQuery( sql: "SELECT * FROM " + TABLE_NAME, selectionArgs: null);
    }
    no usages
    public boolean updateData(String id, String name, String marks) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues contentValues = new ContentValues();
        contentValues.put(COL_1, id);
        contentValues.put(COL_2, name);
        contentValues.put(COL_3, marks);
        db.update(TABLE_NAME, contentValues, whereClause: "ID = ?", new String[]{id});
        return true;
    }
    no usages
    public Integer deleteData(String id) {
        SQLiteDatabase db = this.getWritableDatabase();
        return db.delete(TABLE_NAME, whereClause: "ID = ?", new String[]{id});
    }
}
```

activity_main.xml

```xml
activity_main.xml   MainActivity.java   DatabaseHelper.java

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <LinearLayout
        android:orientation="vertical"
        android:padding="16dp"
        xmlns:android="http://schemas.android.com/apk/res/android"
        android:layout_width="match_parent"
        android:layout_height="match_parent">
        <EditText android:id="@+id/editName"
            android:hint="Enter Name"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"/>
        <EditText android:id="@+id/editMarks"
            android:hint="Enter Marks"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"/>
```

```
        <EditText android:id="@+id/editId"
            android:hint="Enter ID (for Update/Delete)"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"/>
        <Button android:id="@+id/btnAdd"
            android:text="Add Data"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"/>
        <Button android:id="@+id/btnView"
            android:text="View Data"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"/>
        <Button android:id="@+id/btnUpdate"
            android:text="Update Data"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"/>
        <Button android:id="@+id/btnDelete"
            android:text="Delete Data"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"/>
    </LinearLayout>

</androidx.constraintlayout.widget.ConstraintLayout>
```

MainActivity.java

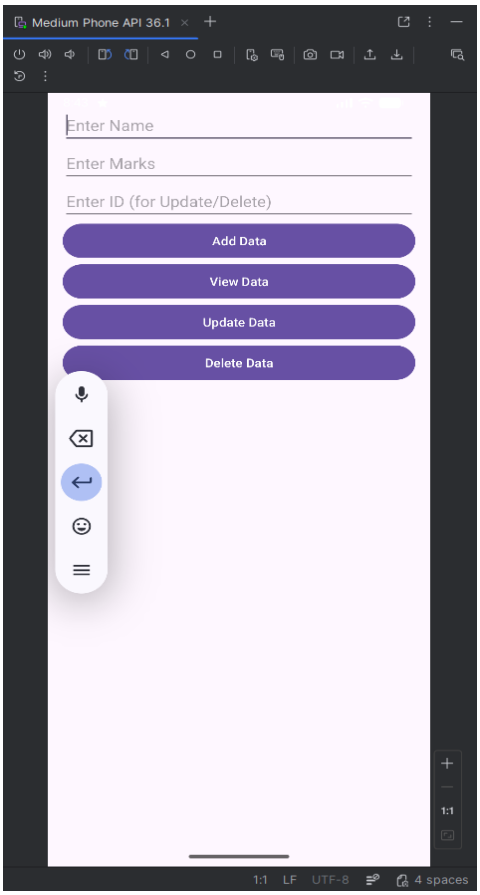
activity_main.xml | MainActivity.java | DatabaseHelper.java

```
package com.example.experiment_11;

import android.os.Bundle;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
import android.database.Cursor;
import android.view.View;
import android.widget.*;

public class MainActivity extends AppCompatActivity {
    5 usages
    DatabaseHelper myDb;
    3 usages
    EditText editName, editMarks, editId;
    2 usages
    Button btnAdd, btnView, btnUpdate, btnDelete;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        myDb = new DatabaseHelper( context: this);

        editName = findViewById(R.id.editName);
        editMarks = findViewById(R.id.editMarks);
        editId = findViewById(R.id.editId);

        btnAdd = findViewById(R.id.btnAdd);
```

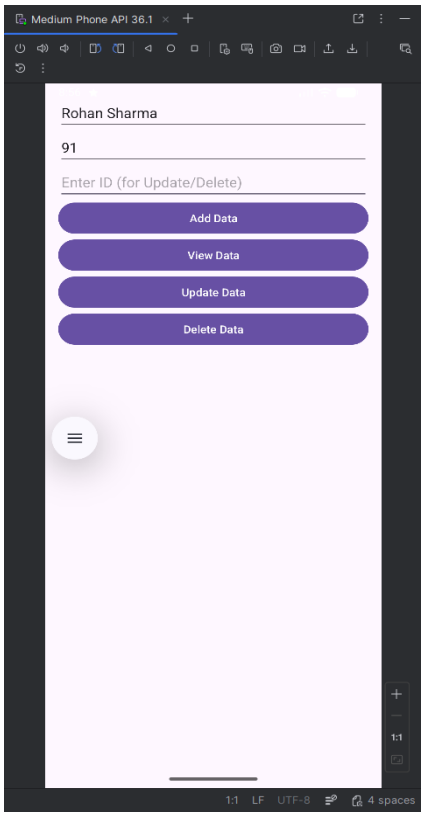
```java
        btnView = findViewById(R.id.btnView);
        btnUpdate = findViewById(R.id.btnUpdate);
        btnDelete = findViewById(R.id.btnDelete);

        // Add Data
        btnAdd.setOnClickListener( View v -> {
            boolean isInserted = myDb.insertData(
                    editName.getText().toString(),
                    editMarks.getText().toString()
            );

            Toast.makeText( context: this,
                    isInserted ? "Data Inserted" : "Failed",
                    Toast.LENGTH_SHORT).show();
        });

        // View Data
        btnView.setOnClickListener( View v -> {
            Cursor res = myDb.getAllData();

            if (res.getCount() == 0) {
                showMessage( title: "Error",  message: "No Data Found");
                return;
            }

            StringBuilder buffer = new StringBuilder();
            while (res.moveToNext()) {
                buffer.append("ID: ").append(res.getString( columnIndex: 0)).append("\n");
                buffer.append("Name: ").append(res.getString( columnIndex: 1)).append("\n");
                buffer.append("Marks: ").append(res.getString( columnIndex: 2)).append("\n\n");
            }

            showMessage( title: "Data", buffer.toString());
        });
```

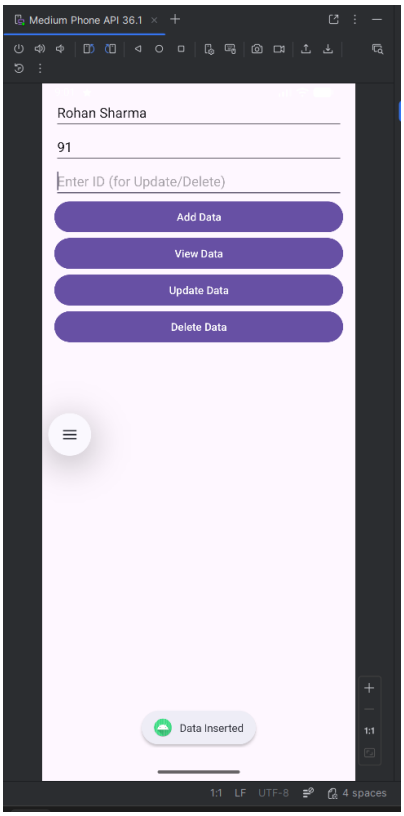
```java
    // Update
        btnUpdate.setOnClickListener( View v -> {
            boolean isUpdated = myDb.updateData(
                    editId.getText().toString(),
                    editName.getText().toString(),
                    editMarks.getText().toString()
            );
            Toast.makeText( context: this,
                    isUpdated ? "Updated" : "Failed",
                    Toast.LENGTH_SHORT).show();
        });
    // Delete
        btnDelete.setOnClickListener( View v -> {
            int deletedRows = myDb.deleteData(editId.getText().toString());
            Toast.makeText( context: this,
                    deletedRows > 0 ? "Deleted" : "Not Found",
                    Toast.LENGTH_SHORT).show();
        });
    }
    2 usages
    public void showMessage(String title, String message) {
        new android.app.AlertDialog.Builder( context: this)
                .setTitle(title)
                .setMessage(message)
                .show();
    }
}
```

Output:

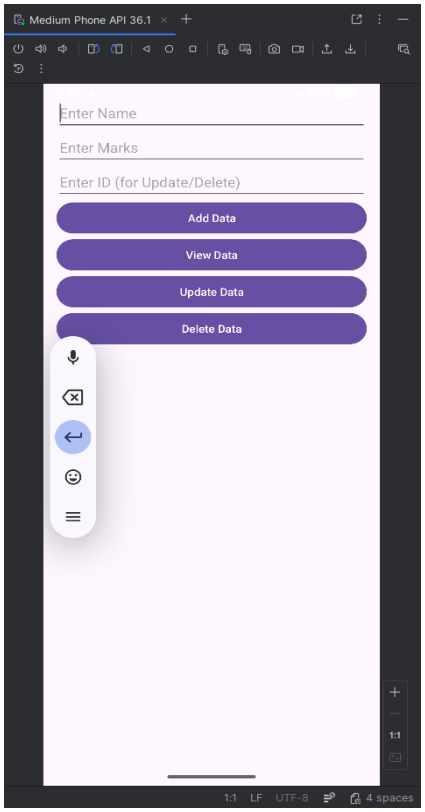

Medium Phone API 36.1
Enter Name
Enter Marks
Enter ID (for Update/Delete)
Add Data
View Data
Update Data
Delete Data
1:1
1:1 LF UTF-8 4 spaces

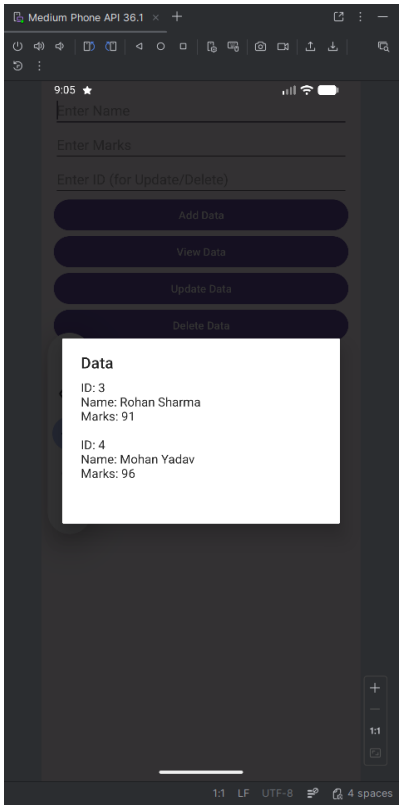
1) Insert Operation



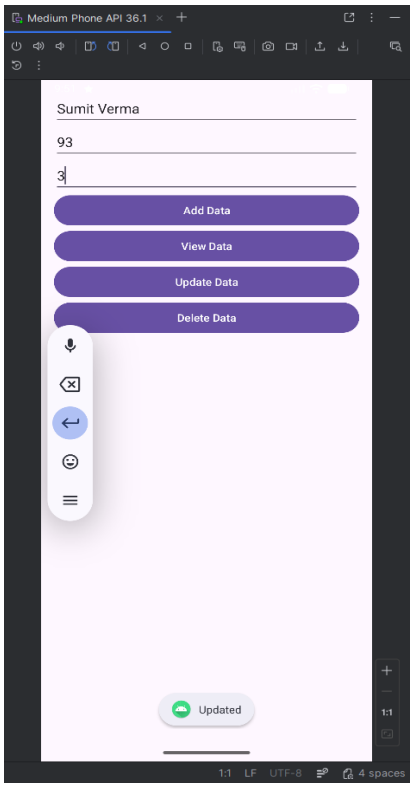


2) Read (or View) Operation

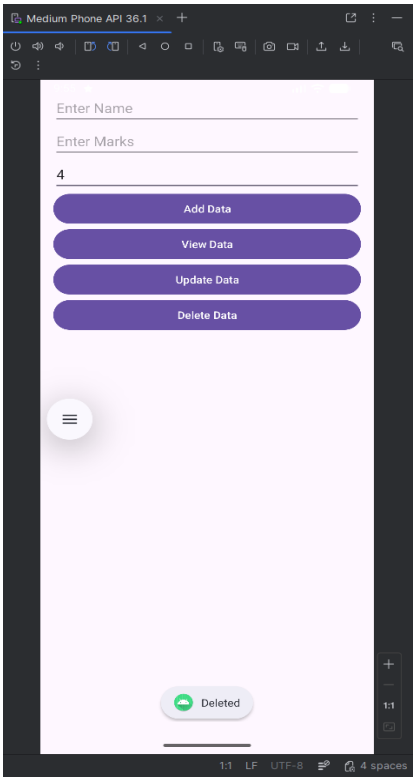


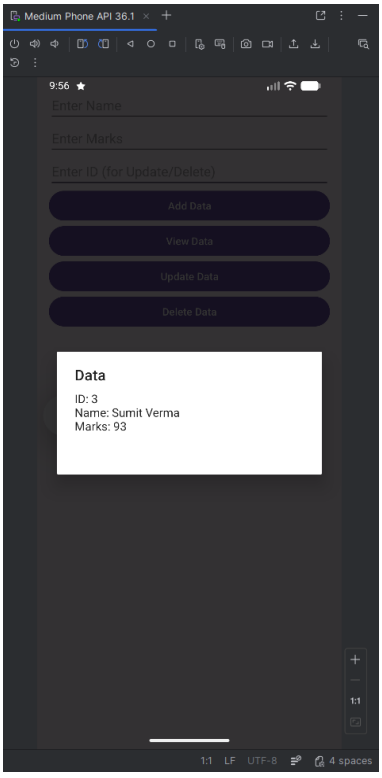


3) Update Operation



4) Delete Operation





Conclusion:

This experiment demonstrated how to integrate and use an SQLite database in an Android application for local data storage. It provided hands-on experience in creating tables and performing basic CRUD operations such as insert, update, delete, and retrieve. Overall, it strengthened understanding of efficient data management within Android apps.